

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №9
по курсу «Логика и основы алгоритмизации в
инженерных задачах»
на тему «Поиск расстояний в графе»

Выполнил:

студенты группы 24ВВВ2

Тремасов К. И.

Бортнев А. О.

Принял:

к.т.н. доцент Юрова О.В.

к.т.н. Деев М.В.

Пенза 2025

Название

Поиск расстояний в графе.

Цель работы

Научиться искать расстояние между вершинами в графе.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++
3. *Реализуйте процедуру поиска расстояний для графа, представленного списками смежности.

Задание 2*

1. *Реализуйте процедуру поиска расстояний на основе обхода в глубину
2. *Реализуйте процедуру поиска расстояний на основе обхода в глубину для графа, представленного списками смежности
3. *Оцените время работы реализаций алгоритмов поиска расстояний на основе обхода в глубину и обхода в ширину для графов разных порядков.

Описание метода решения задачи

Метод решения основан на алгоритмах обхода графов в ширину (BFS) и глубину (DFS) с использованием двух различных структур данных для представления графа: матрицы смежности и списков смежности.

Алгоритм BFS реализует классический поиск в ширину с использованием очереди, где обход начинается со стартовой вершины и последовательно охватывает все вершины на текущем уровне расстояния перед переходом к следующим уровням. Для матричного представления поиск смежных вершин требует полного перебора всех вершин в строке матрицы, что дает сложность $O(V)$ для каждой вершины, где V - количество вершин. В случае списков смежности доступ к соседним вершинам осуществляется напрямую, что обеспечивает сложность $O(k)$, где k - количество соседей вершины.

Алгоритм DFS использует стек для обхода в глубину, где происходит движение по одному пути до его полного исследования с последующим возвратом. В матричной реализации проверка соседей также требует полного перебора строки матрицы, тогда как списковая реализация позволяет эффективно итерироваться только по фактическим соседям вершины.

Во всех случаях BFS показывается более лучшее время поиска расстояния в сравнении с DFS.

Листинг

```

#include <iostream>
#include <vector>
#include <queue>
#include <stack>
#include <random>
#include <iomanip>
#include <limits>
#include <chrono>
using namespace std;
using namespace std::chrono;
class Graph {
private:
    vector<vector<int>> adjacencyMatrix;
    vector<vector<int>> adjacencyList;
    int vertices;

    void buildAdjacencyListFromMatrix() {
        adjacencyList.clear();
        adjacencyList.resize(vertices);
        for (int i = 0; i < vertices; i++) {
            for (int j = 0; j < vertices; j++) {
                if (adjacencyMatrix[i][j] == 1) {
                    adjacencyList[i].push_back(j);
                }
            }
        }
    }

public:
    Graph(int n) : vertices(n) {
        generateRandomGraph();
    }
    void generateRandomGraph() {
        random_device rd;
        mt19937 gen(rd());
        uniform_int_distribution<> dis(0, 1);
        adjacencyMatrix.resize(vertices, vector<int>(vertices, 0));
        for (int i = 0; i < vertices; i++) {
            for (int j = i + 1; j < vertices; j++) {
                int edge = dis(gen);
                adjacencyMatrix[i][j] = edge;
                adjacencyMatrix[j][i] = edge;
            }
        }
        buildAdjacencyListFromMatrix();
    }
    void printAdjacencyMatrix() {
        cout << "\nМатрица смежности:\n";
        cout << "    ";
        for (int i = 0; i < vertices; i++) {
            cout << setw(3) << i;
        }
        cout << "\n";
        for (int i = 0; i < vertices; i++) {
            cout << setw(3) << i;
            for (int j = 0; j < vertices; j++) {
                cout << setw(3) << adjacencyMatrix[i][j];
            }
            cout << "\n";
        }
    }
    void printAdjacencyList() {
        cout << "\nСписки смежности:\n";
        for (int i = 0; i < vertices; i++) {
            cout << i << ": ";
            for (int neighbor : adjacencyList[i]) {

```

```

        cout << neighbor << " ";
    }
    cout << "\n";
}
}
vector<int> bfsDistanceMatrix(int startVertex) {
    auto start = high_resolution_clock::now();
    vector<int> dist(vertices, -1);
    queue<int> q;
    dist[startVertex] = 0;
    q.push(startVertex);
    while (!q.empty()) {
        int current = q.front();
        q.pop();
        for (int i = 0; i < vertices; i++) {
            if (adjacencyMatrix[current][i] == 1 && dist[i] == -1) {
                dist[i] = dist[current] + 1;
                q.push(i);
            }
        }
    }
    auto end = high_resolution_clock::now();
    auto duration = duration_cast<microseconds>(end - start);
    cout << "Время выполнения BFS (матрица): " << duration.count() << "
мкс\n";
    return dist;
}
vector<int> bfsDistanceList(int startVertex) {
    auto start = high_resolution_clock::now();
    vector<int> dist(vertices, -1);
    queue<int> q;
    dist[startVertex] = 0;
    q.push(startVertex);
    while (!q.empty()) {
        int current = q.front();
        q.pop();
        for (int neighbor : adjacencyList[current]) {
            if (dist[neighbor] == -1) {
                dist[neighbor] = dist[current] + 1;
                q.push(neighbor);
            }
        }
    }
    auto end = high_resolution_clock::now();
    auto duration = duration_cast<microseconds>(end - start);
    cout << "Время выполнения BFS (списки): " << duration.count() << " мкс\n";
    return dist;
}
vector<int> dfsDistanceMatrix(int startVertex) {
    auto start = high_resolution_clock::now();
    vector<int> dist(vertices, -1);
    stack<int> s;
    dist[startVertex] = 0;
    s.push(startVertex);
    while (!s.empty()) {
        int current = s.top();
        s.pop();
        for (int i = 0; i < vertices; i++) {
            if (adjacencyMatrix[current][i] == 1 && dist[i] == -1) {
                dist[i] = dist[current] + 1;
                s.push(i);
            }
        }
    }
    auto end = high_resolution_clock::now();
    auto duration = duration_cast<microseconds>(end - start);

```

```

        cout << "Время выполнения DFS (матрица): " << duration.count() << "
мкс\n";
        return dist;
    }

    vector<int> dfsDistanceList(int startVertex) {
        auto start = high_resolution_clock::now();
        vector<int> dist(vertices, -1);
        stack<int> s;
        dist[startVertex] = 0;
        s.push(startVertex);
        while (!s.empty()) {
            int current = s.top();
            s.pop();
            for (int neighbor : adjacencyList[current]) {
                if (dist[neighbor] == -1) {
                    dist[neighbor] = dist[current] + 1;
                    s.push(neighbor);
                }
            }
        }
        auto end = high_resolution_clock::now();
        auto duration = duration_cast<microseconds>(end - start);
        cout << "Время выполнения DFS (списки): " << duration.count() << " мкс\n";
        return dist;
    }

    void printDistances(const vector<int>& dist, int startVertex) {
        cout << "\nРасстояния от вершины " << startVertex << ":\n";
        for (int i = 0; i < vertices; i++) {
            cout << "Вершина " << i << ": ";
            if (dist[i] == -1) {
                cout << "недостижима";
            }
            else {
                cout << dist[i];
            }
            cout << "\n";
        }
    }

};

int main() {
    setlocale(LC_ALL, "Russian");
    int vertices;
    cout << "Введите количество вершин графа: ";
    cin >> vertices;
    if (vertices <= 0) {
        cout << "Неверное количество вершин! Установлено значение по умолчанию: 5"
<< endl;
        vertices = 5;
    }
    Graph graph(vertices);
    int choice;
    do
    {
        cout << "\n1. Установить количество вершин";
        cout << "\n2. Показать матрицу смежности";
        cout << "\n3. Показать списки смежности";
        cout << "\n4. BFS - расстояния (матрица смежности)";
        cout << "\n5. BFS - расстояния (списки смежности)";
        cout << "\n6. DFS - расстояния (матрица смежности)";
        cout << "\n7. DFS - расстояния (списки смежности)";
        cout << "\n0. Выход";
        cout << "\nВыберите операцию: ";
        cin >> choice;
        switch (choice) {
            case 1: {
                cout << "Введите количество вершин: ";

```

```

    cin >> vertices;
    if (vertices > 0) {
        graph = Graph(vertices);
    }
    else {
        cout << "Неверное количество вершин!\n";
    }
    break;
}
case 2: {
    graph.printAdjacencyMatrix();
    break;
}
case 3: {
    graph.printAdjacencyList();
    break;
}
case 4: {
    int startVertex;
    cout << "Введите начальную вершину: ";
    cin >> startVertex;
    if (startVertex >= 0 && startVertex < vertices) {
        vector<int> dist = graph.bfsDistanceMatrix(startVertex);
        graph.printDistances(dist, startVertex);
    }
    else {
        cout << "Неверная вершина!\n";
    }
    break;
}
case 5: {
    int startVertex;
    cout << "Введите начальную вершину: ";
    cin >> startVertex;
    if (startVertex >= 0 && startVertex < vertices) {
        vector<int> dist = graph.bfsDistanceList(startVertex);
        graph.printDistances(dist, startVertex);
    }
    else {
        cout << "Неверная вершина!\n";
    }
    break;
}
case 6: {
    int startVertex;
    cout << "Введите начальную вершину: ";
    cin >> startVertex;
    if (startVertex >= 0 && startVertex < vertices) {
        vector<int> dist = graph.dfsDistanceMatrix(startVertex);
        graph.printDistances(dist, startVertex);
    }
    else {
        cout << "Неверная вершина!\n";
    }
    break;
}
case 7: {
    int startVertex;
    cout << "Введите начальную вершину: ";
    cin >> startVertex;
    if (startVertex >= 0 && startVertex < vertices) {
        vector<int> dist = graph.dfsDistanceList(startVertex);
        graph.printDistances(dist, startVertex);
    }
    else {
        cout << "Неверная вершина!\n";
    }
}

```

```

        break;
    }
    case 0: {
        break;
    }
    default: {
        cout << "Неверный выбор!\n";
        break;
    }
}
} while (choice != 0);
return 0;
}

```

Текст к программе

Данная программа реализует алгоритмы обхода графов BFS и DFS с использованием двух структур данных: матрицы смежности и списков смежности. При тестировании программы были выявлены четкие закономерности в производительности. Программа наглядно демонстрирует важность выбора оптимальной структуры данных для работы с графами, где списки смежности обеспечивают более эффективный обход за счет прямого доступа к смежным вершинам без избыточных проверок.

Результаты работы программы

```

C:\Users\student\source\repos\9lb\64\Debug\9lb.exe
Введите количество вершин графа: 6
1. Установить количество вершин
2. Показать матрицу смежности
3. Показать списки смежности
4. BFS - расстояния (матрица смежности)
5. BFS - расстояния (списки смежности)
6. DFS - расстояния (матрица смежности)
7. DFS - расстояния (списки смежности)
8. Выход
Выберите операцию: 2
Матрица смежности:
  0 1 2 3 4 5
0 0 1 0 1 0
1 0 0 1 0 0
2 0 1 0 1 0 0
3 1 0 1 0 0 0
4 1 1 0 0 0 0
5 0 0 0 0 0 0
1. Установить количество вершин
2. Показать матрицу смежности
3. Показать списки смежности
4. BFS - расстояния (матрица смежности)
5. BFS - расстояния (списки смежности)
6. DFS - расстояния (матрица смежности)
7. DFS - расстояния (списки смежности)
8. Выход
Выберите операцию:

```

```

Выбрать C:\Users\student\source\repos\9lb\x64\Debug\9lb.exe

Матрица смежности:
  0 1 2 3 4 5
0 0 0 0 1 1 0
1 0 0 1 0 1 0
2 0 1 0 1 0 0
3 1 0 1 0 0 0
4 1 1 0 0 0 0
5 0 0 0 0 0 0

1. Установить количество вершин
2. Показать матрицу смежности
3. Показать списки смежности
4. BFS - расстояния (матрица смежности)
5. BFS - расстояния (списки смежности)
6. DFS - расстояния (матрица смежности)
7. DFS - расстояния (списки смежности)
8. Выход
Выберите операцию: 4
Введите начальную вершину: 5
Время выполнения BFS (матрица): 14 мкс

Расстояния от вершины 5:
Вершина 0: недостижима
Вершина 1: недостижима
Вершина 2: недостижима
Вершина 3: недостижима
Вершина 4: недостижима
Вершина 5: 0

```

```

Консоль отладки Microsoft Visual Studio

6. DFS - расстояния (матрица смежности)
7. DFS - расстояния (списки смежности)
8. Выход
Выберите операцию: 6
Введите начальную вершину: 3
Время выполнения DFS (матрица): 17 мкс

Расстояния от вершины 3:
Вершина 0: 1
Вершина 1: 2
Вершина 2: 1
Вершина 3: 0
Вершина 4: 3
Вершина 5: недостижима

1. Установить количество вершин
2. Показать матрицу смежности
3. Показать списки смежности
4. BFS - расстояния (матрица смежности)
5. BFS - расстояния (списки смежности)
6. DFS - расстояния (матрица смежности)
7. DFS - расстояния (списки смежности)
8. Выход
Выберите операцию: 0

C:\Users\student\source\repos\9lb\x64\Debug\9lb.exe (процесс 4988) завершил работу с кодом 0.
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" -> "Параметры" -> "Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно...

```

Выводы

В ходе лабораторной работы было установлено, что алгоритм BFS демонстрирует более высокую эффективность по сравнению с DFS при решении задачи поиска расстояний в графе. BFS обеспечивает систематический обход вершин по уровням, что гарантирует нахождение кратчайшего пути до каждой достижимой вершины. В отличие от него, DFS может исследовать более длинные пути прежде, чем вернуться к ближайшим соседям, что делает его менее оптимальным для данной задачи. При использовании одинаковых структур данных BFS показал сопоставимое или лучшее время выполнения, подтвердив свою целесообразность для задач поиска расстояний. Наилучшие результаты показала комбинация BFS со списками смежности, которая сочетает эффективность алгоритма с оптимальной структурой хранения графа.